

ZOHO SETU PROJECT

SETU PROJECT

Baseline kernel OS

Software Domain

Description

Create a minimal 64-bit microkernel with core components like USB drivers, memory management, and a graphical shell.

Project Type: Linux

Category: Software Based

Atharva Bodade

Class: Third Year

Branch: Information Technology

Shri Sant Gajanan Maharaj College of Engineering, Shegaon

GitHub: <https://github.com/UnsettledAverage73/zoho-os>

Reference: <https://kernel.org/>

May 2026

Contents

1	Vision & Problem Statement	2
1.1	The Limitations of Today	2
1.2	Project Philosophy	2
2	Architecture Overview	3
2.1	System Architecture Diagram	3
2.2	Structural Integrity	3
3	Boot Process	5
3.1	Bootstrapping 64-bit Mode	5
3.2	Key Milestones	5
4	Core Kernel Systems	6
4.1	Physical Memory Manager (PMM)	6
4.2	Virtual Memory Manager (VMM)	6
4.3	Interrupt Handling & IDT	6
4.4	Virtual File System (VFS)	6
4.5	Observability (KStats & KTrace)	7
4.6	Terminal Abstraction (TTY)	7
4.7	USB Support (XHCI)	7
5	Unique Features	8
5.1	Interactive Shell	8
5.2	Video Demonstration	8
5.3	Real-time System Monitor	8
5.4	GUI Performance Optimization	8
6	Technical Challenges	9
6.1	Standardizing Framebuffer Pitch	9
6.2	The Huge Page Corruption Bug	9
6.3	Memory Overlap (OOM)	9
7	Future Roadmap	10
8	System State	11
8.1	Terminal Output	11

Chapter 1

Vision & Problem Statement

The goal of this project is to develop a minimal, educational Linux-type kernel from scratch. It serves as a baseline for understanding low-level systems programming, memory management, and driver development.

1.1 The Limitations of Today

Modern kernels like Linux are incredibly complex, making it difficult for students and researchers to grasp the core fundamentals of how an OS interacts with hardware. This project aims to strip away the complexity to provide a clear, functional baseline.

1.2 Project Philosophy

Baseline kernel OS is built to be a minimal yet extensible system. This requires:

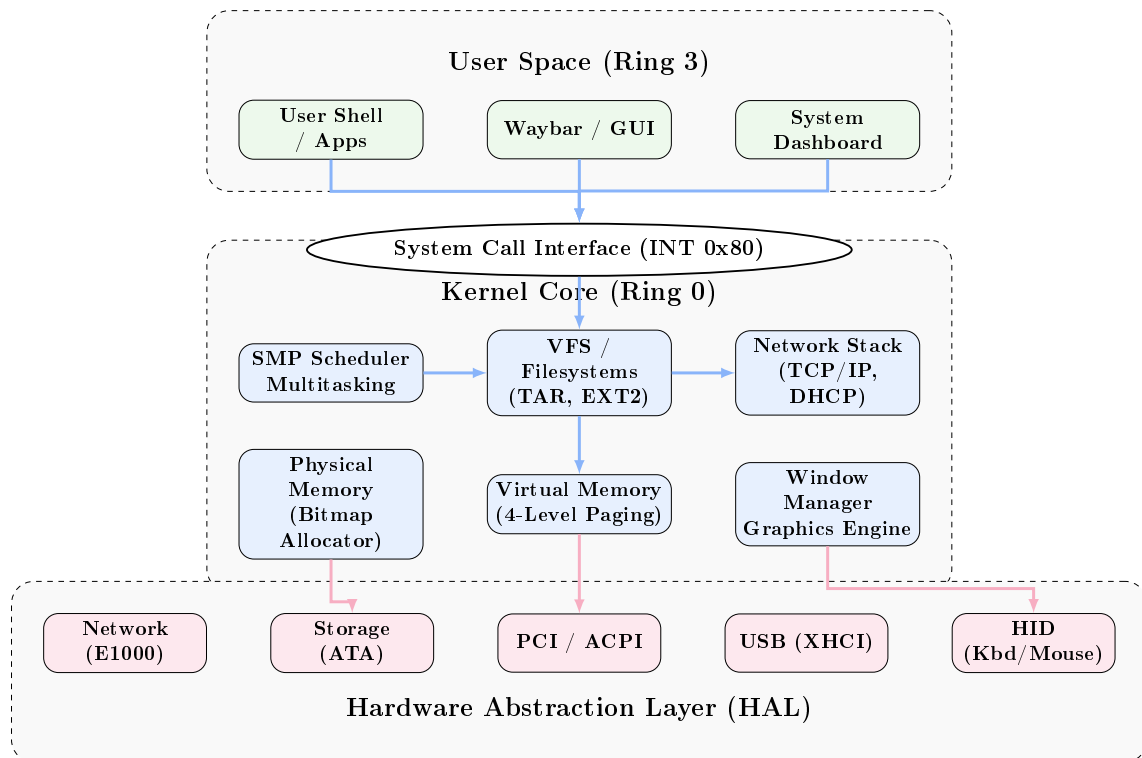
- **Educational Clarity:** Well-documented code that maps directly to architectural concepts.
- **Hardware Abstraction:** A clean interface between the kernel and the underlying x86_64 hardware.
- **Extensibility:** A modular design that allows for easy addition of new drivers and system calls.

Chapter 2

Architecture Overview

Baseline kernel OS implements a ****Modular Monolithic Architecture****. While all core services run within the same address space in Ring 0 for performance, the system is logically divided into strictly defined layers and modules to ensure maintainability and structural clarity.

2.1 System Architecture Diagram



2.2 Structural Integrity

The architecture is designed around three primary pillars:

- **Isolation:** User-space applications are strictly isolated from the kernel and each other via 4-level paging and hardware-enforced protection rings.

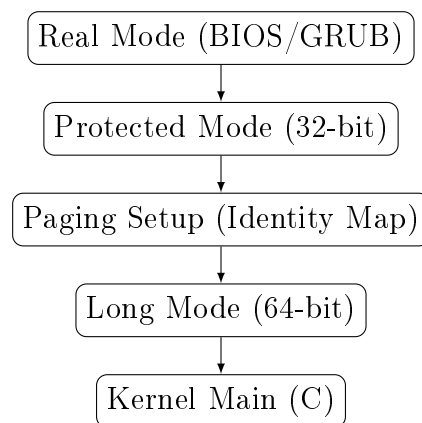
- **Abstraction:** The VFS and HAL provide a consistent interface for higher-level systems, allowing hardware-specific details to be swapped or updated without affecting the core logic.
- **Observability:** Integrated logging (`klog`), statistics (`kstats`), and tracing (`ktrace`) are built into the kernel core to facilitate real-time debugging and performance monitoring.

Chapter 3

Boot Process

The transition from a powered-off state to a running 64-bit environment involves a complex handoff between assembly and C.

3.1 Bootstrapping 64-bit Mode



3.2 Key Milestones

1. **Multiboot2 Header:** Allows GRUB to recognize the kernel.
2. **PAE Paging:** Necessary to address the 64-bit space.
3. **GDT Flush:** Loading the global descriptor table for code and data segments.

Chapter 4

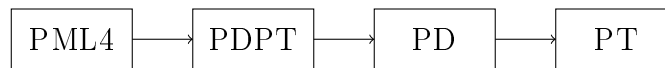
Core Kernel Systems

4.1 Physical Memory Manager (PMM)

The PMM uses a stack-based frame allocator combined with a bitmap.

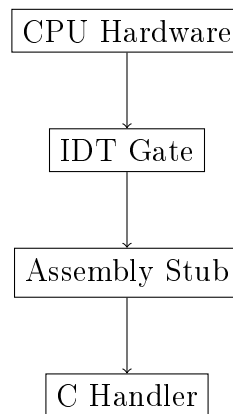
4.2 Virtual Memory Manager (VMM)

Baseline kernel OS implements a 4-level paging system.



4.3 Interrupt Handling & IDT

Baseline kernel OS manages hardware asynchronous events through a centralized Interrupt Descriptor Table (IDT).



4.4 Virtual File System (VFS)

The VFS layer abstracts underlying storage formats (TAR, EXT2) into a unified tree structure.

- **Disk-backed TAR:** Used for the initial ramdisk and system applications.
- **Device Nodes:** Support for `/dev/tty` and `/dev/serial`.

4.5 Observability (KStats & KTrace)

Baseline kernel OS includes built-in observability tools that track kernel performance from the earliest boot stages.

- **KStats:** Monitors physical memory usage, uptime, and active task counts.
- **KTrace:** Provides an event buffer for high-speed tracing of kernel execution paths, essential for debugging race conditions in SMP mode.

4.6 Terminal Abstraction (TTY)

The TTY layer provides a unified interface for text-based communication, multiplexing output between the local VGA framebuffer and the remote Serial (COM1) port.

4.7 USB Support (XHCI)

Baseline kernel OS features an eXtensible Host Controller Interface (XHCI) driver foundation, enabling high-speed communication with USB 3.0 devices such as human interface devices (HID) and mass storage.

Chapter 5

Unique Features

5.1 Interactive Shell

Baseline kernel OS features a userland shell that supports basic file operations and system queries.

5.2 Video Demonstration

A full demonstration of the Baseline kernel OS boot process, kernel initialization, and interactive shell is available on YouTube.

[\[Watch the Baseline kernel OS Demo on YouTube\]](#)

5.3 Real-time System Monitor

The Dashboard task provides live CPU and Memory statistics. It runs as a separate kernel task, demonstrating the scheduler's ability to handle background observability.

5.4 GUI Performance Optimization

The window manager uses a "Dirty Rectangle" algorithm to minimize redraw overhead, combined with a dedicated kernel task to ensure smooth rendering even under heavy kernel load.

Chapter 6

Technical Challenges

6.1 Standardizing Framebuffer Pitch

Problem: Initial graphics implementation resulted in a black screen or skewed output on certain virtualized hardware. **Discovery:** I found that hardware often uses a ‘pitch’ (bytes per scanline) that is wider than the visible width. **Resolution:** Refactored all graphics primitives (`clear`, `blit`, `put_pixel`) to use `fb_pitch` for coordinate calculations instead of simple `width * 4`.

6.2 The Huge Page Corruption Bug

An `INT 6` fault occurred because the VMM treated 2MB Huge Pages as tables. **Resolution:** Refactored the identity map to use 4KB pages and updated VMM to handle flag propagation correctly. This ensures that 4KB mappings can coexist with the initial identity map without corrupting executable code.

6.3 Memory Overlap (OOM)

The heap and PMM were conflicting on the same physical range. **Resolution:** Explicitly reserved the 128MB heap range in the PMM bitmap.

Chapter 7

Future Roadmap

- GPU Acceleration via VirtIO.
- Custom Ext2/Ext4 Filesystem.
- Distributed Runtime for AI Inference.

Chapter 8

System State

8.1 Terminal Output

```
[INFO ] TTY: Initializing TTY subsystem...
[INFO ] TTY: TTY subsystem initialized.
[INFO ] KERNEL: Baseline kernel OS Booting...
PMM Initialized. Total free frames: Done.
[INFO ] KMALLOC: Kernel heap initialized at 16MB (Size: 16MB)
[INFO ] KSTATS: Initializing kernel statistics...
[INFO ] KSTATS: Kernel statistics initialized.
[INFO ] KTRACE: Initializing kernel tracing...
[INFO ] KTRACE: Kernel tracing initialized.
[INFO ] ACPI: Initializing ACPI...
[INFO ] PCI: Enumerating PCI devices...
[INFO ] XHCI: Initializing XHCI (USB 3.0) controller...
[INFO ] XHCI: XHCI subsystem initialized (stub).
[INFO ] NET: Network stack initialized with IP 10.0.2.15

Baseline kernel OS Interactive Shell
zoho> ls
bin/
bin/shell
test.txt
```

Contact & Links

Atharva Bodade

atharvabodade@gmail.com

ssgm_308217@ssgmce.ac.in

[GitHub Profile](#)

[LinkedIn Profile](#)

[Project Reference](#)